

National Taiwan Normal University
CSIE Computer Programming I

Instructor: Po-Wen Chi
Due Date: 2024.12.03 PM 11:59

Assignment 4

Policies:

- Zero tolerance for late submission.
- Please pack all your submissions in one zip file. **RAR is not allowed!!**
- For convenience, your executable programs must be named following the rule hw**XXYY**, where the red part is the homework number and the blue part is the problem number. For example, **hw0102** is the executable program for homework #1 problem 2.
- I only accept **PDF** or **TEXT**. MS Word is not allowed.
- **Do not forget your Makefile. For convenience, each assignment needs only one Makefile.**
- Please provide a README file. The README file should have at least the following information:
 - Your student ID and your name.
 - How to build your code.
 - How to execute your built programs.
 - Anything that you want to notify our TAs.
- **DO NOT BE A COPYCAT!!** You will get ZERO if you are caught.

4.1 Big Two (20 pts)

Have you ever played the poker game **Big Two**? As the game's name, the point order in this game is

$$2 > A > K > Q > J > 10 > 9 > 8 > 7 > 6 > 5 > 4 > 3$$

In this problem, I want you to sort cards for Big Two players. Again, the card encoding is as follows:

- 1-13: ♠ A, 2, ..., 9, 10, J, Q, K
- 14-26: ♥ A, 2, ..., 9, 10, J, Q, K
- 27-39: ♦ A, 2, ..., 9, 10, J, Q, K
- 40-52: ♣ A, 2, ..., 9, 10, J, Q, K

The sorting rule is here:

- Ascending order.
- For the same point, the order should be ♠ → ♥ → ♦ → ♣.

You need to implement the following function in **poker.c** with a header file **poker.h**.

```
1 // I promise the size of array is 13, so I skip the size input.
2 int32_t big_two_sort( int8_t cards[] );
```

For your convenience, in this problem, I guarantee the element number of the input array is 13. If the input is invalid, please return -1; otherwise, return 0. The TA will prepare **hw0401.c** for you and I promise that **poker.h** will be included. You must build **hw0401.c** to **hw0401** in your Makefile.

4.2 Mahjong (20 pts)

OK, OK, OK. I know you do not like poker games. So I change the topic. Let's play Mahjong. For your simplicity, figure 4.1 to see what Mahjong is.



FIGURE 4.1: 麻將。

		Numbers									
		1	2	3	4	5	6	7	8	9	
Simples	Dots										1 - 9
	Bamboo										10 - 18
	Characters										19 - 27
Winds						Dragons					
East	South	West	North			Red	Green	White			
				28-31					32-34		

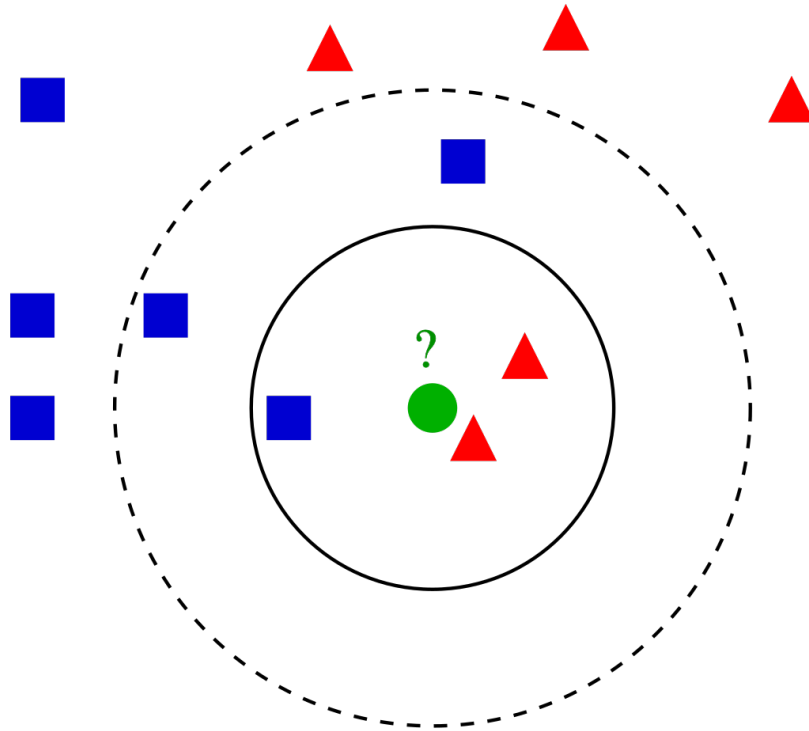


FIGURE 4.3: A k -NN example.

4.3 k -NN Classifier (20 pts)

In statistics, the k -nearest neighbors algorithm (k -NN) is a non-parametric supervised learning method first developed by Evelyn Fix and Joseph Hodges in 1951, and later expanded by Thomas Cover. It is used for classification and regression. In both cases, the input consists of the k closest training examples in a data set. This time, I want you to develop a k -NN classifier library. Undoubtedly, you need to know how the k -NN classifier works.

In k -NN classification, the output is a class membership. An object is classified by a plurality vote of its neighbors, with the object being assigned to the class most common among its k nearest neighbors (k is a **positive integer**, typically small). If $k = 1$, then the object is simply assigned to the class of that single nearest neighbor.

Figure 4.3 is an example. The test sample (green dot) should be classified either to blue squares or to red triangles. If $k = 3$ (solid line circle) it is assigned to the red triangles because there are 2 triangles and only 1 square inside the inner circle. If $k = 5$ (dashed line circle) it is assigned to the blue squares (3 squares vs. 2 triangles inside the outer circle).

```

1 // Input the dataset.
2 // size is the element number of label[], x[], y[].
3 // size range: 10-10000

```

```

4 // If size is out of range, return -1; otherwise, return 0.
5 int32_t inputDataset( uint32_t size, int32_t label[], int32_t x
    [], int32_t y[] );
6
7 // Given a test sample (x,y) and return its class.
8 // k is for k-NN.
9 // Note that if (x,y) is in the input dataset, directly return
    the input label.
10 // if k is invalid, return -1; otherwise, return its class.
11 int32_t classify( int32_t k, int32_t x, int32_t y );

```

You should also prepare a header file called **knn.h**. Our TAs will prepare hw0403.c which includes knn.h and uses these functions. **Do not forget to make hw0403.c to hw0403 in your Makefile.**

4.4 Bingo (20 pts)

Have you ever played Bingo? This time, I want you to develop a Bingo game for a player and a computer. The rule is in the following link. Note that in this problem, you only need to develop Taiwanese Bingo.

<https://zh.wikipedia.org/zh-tw/%E7%BE%8E%E5%BC%8F%E8%B3%93%E6%9E%9C>

```

1 $ ./hw0404
2 Setup 01: 1,2
3 Setup 02: 1,1
4 ...
5 Setup 25: 1,3
6 ===
7 02 01 25 03 04 | xx xx xx xx xx
8 06 07 08 09 10 | xx xx xx xx xx
9 05 11 12 13 14 | xx xx xx xx xx
10 15 16 19 17 18 | xx xx xx xx xx
11 20 21 22 23 24 | xx xx xx xx xx
12 ===
13 Player's choice: 01
14 ===
15 02 01 25 03 04 | xx xx xx xx xx
16 06 07 08 09 10 | xx xx xx xx xx
17 05 11 12 13 14 | xx xx xx xx xx
18 15 16 19 17 18 | xx xx xx 01 xx
19 20 21 22 23 24 | xx xx xx xx xx
20 ===
21 Computer's choice: 02
22 ===
23 ...
24 ===
25 The winner is Player.

```

The setup location format is r, c where r is for row and c is for column. $1 \leq r, c \leq 5$. For any invalid inputs, ask the player to input the location

again. After the setup phase, the left side is the player's board and the right side is the computer's board. Then the player and the computer will pick a number in turn. The picked number will be colored yellow. When there are three lines on their board, including a horizontal, vertical, or diagonal line, the program will print the winner and terminate itself. If the case is draw, just print "Draw". For any invalid choice, make the player re-choose a number.

4.5 Machine Learning (20 pts)

ML in C

Panda currently has a task: for each student, there are scores for 10 courses of different difficulty levels (0 to 100), and the goal is to convert them into a single value representing the student's level of performance (0 to 1).

He came up with a useful tool called a "Neural Network" and hopes you can help him accomplish this task. You said that you don't know what neural networks are? Don't worry, I will teach you.

Neural Unit

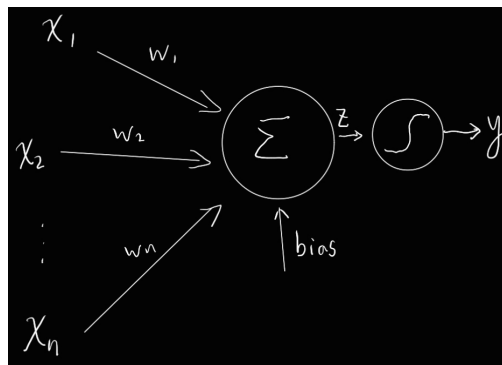


FIGURE 4.4: A Neural Unit

A neural unit basically takes all input values x_i and multiplies each by its corresponding weight w_i (initially, you can set them all 1). Then, it adds a *bias* (initially, you can set it to 0). This value z is then passed through an "activation function" to become the output y .

In mathematical terms, it looks like this:

$$\text{activation} \left(\sum_{i=1}^N (w_i x_i) + \text{bias} \right)$$

Activation Function

The most common activation functions are:

1. ReLU

$$\text{ReLU}(x) = \max(x, 0)$$

Its derivative is 1 when $x > 0$ and 0 otherwise.

2. Sigmoid

$$\theta(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is $\theta(x)(1 - \theta(x))$.

3. tanh

$$\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Its derivative is $\frac{4}{(e^x + e^{-x})^2}$.

In this task, we use the **Sigmoid** function.

$$e = 2.7182818284590452354$$

Neural Network

A neural network consists of multiple layers, each with several neural units. Neural units between layers are interconnected, using the output y from the previous layer as the input x for the current layer.

Here' s an example of a 3×2 network.

Our task is to build a $10 \times 3 \times 1126 \times 42 \times 1$ neural network, where the first layer represents the input x , and the output of the last layer is the predicted result.

Training

Each training session consists of two steps: forward and backward, a method known as backpropagation.

Forward

Compute the output Y from the input X from left to right, recording $\frac{\partial z}{\partial w_i}$ in each neural unit for all i . Don' t worry about the math; here, we can easily verify $\frac{\partial z}{\partial w_i} = x_i$.

After going through all the layers, we obtain y , the predicted answer. We will call it $\tilde{y}$. The actual answer is y .

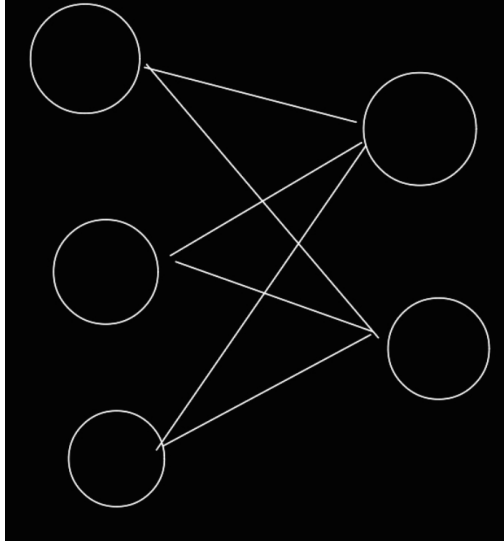


FIGURE 4.5: 3×2 Neural Network

Loss Function We use MSE as the loss function:

$$MSE(y, \tilde{y}) = \frac{(y - \tilde{y})^2}{2}$$

Its derivative with respect to $\tilde{y}$ is $(\tilde{y} - y)$.

Backward

Our goal is to compute $\frac{\partial \text{Loss}}{\partial z}$ for each neural unit. We divide it into two cases:

- Case 1: Output Layer (last layer)

$$\frac{\partial \text{Loss}}{\partial z} = \frac{\partial \text{Loss}}{\partial y} \cdot \frac{\partial y}{\partial z}$$

By the chain rule, we have:

$$\frac{\partial \text{Loss}}{\partial z} = \theta(z)(1 - \theta(z))(\tilde{y} - y)$$

- Case 2: Hidden Layers (excluding the input layer)

$$\frac{\partial \text{Loss}}{\partial z} = \sum \left(\frac{\partial \text{Loss}}{\partial y_i} \cdot w_i \right) \frac{\partial y}{\partial z}$$

where y_i is the output of the next layer. Thus,

$$\frac{\partial \text{Loss}}{\partial y} = \sum w_i \frac{\partial \text{Loss}}{\partial y_i}$$

(I skip the main part of the math)

Afterwards, we obtain $\frac{\partial \text{Loss}}{\partial z}$ for each neural unit. We update W as follows:

$$W'_i = W_i - \text{lr} \times \frac{\partial \text{Loss}}{\partial z} \times y_i$$

The bias B is updated as:

$$B' = B - \text{lr} \times \frac{\partial \text{Loss}}{\partial z}$$

where lr is the learning rate (0.0001) and y_i is the output of this neural unit during this training session.

In this way, we complete one training session. I will provide a small training set for you to test on, but the actual training set will be a large dataset with thousands of entries.

Predict

To predict, simply calculate y using the forward method.

The input-output format for your implementation is listed below.

```
1 $ ./hw0405
2 please input the number of testData: 10
3 testdata 1 X(split by space): 80 60 70 90 100 45 69 71 22 42
4 testdata 1 Y: 0.75
5 testdata 2 X: ...
6 testdata 2 Y: ...
7 ...
8 testdata 10 Y:...
9 input the X you want to predict(split by space):48 60 30 50 10
   45 69 71 22 42
10 the predict Y is 0.1126
```

That hard, right? Don't worry, I prepare a Mandarin version for you.

[learn Neural Network in Computer Programming](#)

4.6 Bonus: Code Explanation (5 pts)

Please explain the following code:

```
1 int p(int i, int N) {
2     return (i < N && printf("%d\n", i) && !p(i + 1, N))
3         || printf("%d\n", i);
4 }
```

Undoubtedly, please use Chinese. Do not tell me that I run this code and I find the output rule. Please explain the code itself.